

Projeto II

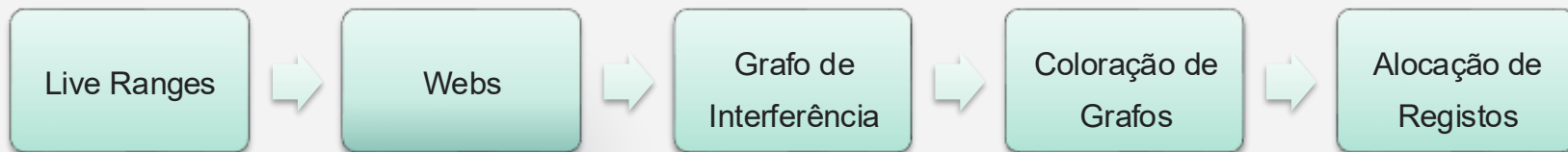
Alocação de Registos por Coloração de Grafos

DA – FEUP 05/2026
Turma 14
Grupo 4

Daniel Rama
Inês Afonso
Pedro Lopes

01 Objetivo

Minimizar o número de registros físicos garantindo que webs interferentes nunca partilham o mesmo registro.



02 Arquitetura do Projeto

Parser:

- Lê e valida os ficheiros de entrada
- Extrai live ranges e número de registos

Web Builder:

- Cria **SubWebs** a partir dos live ranges
- Funde **SubWebs** com Union-Find
- Constrói as **Webs** finais

Allocator:

- Executa os algoritmos de alocação
- Suporta basic, spilling, splitting e free

Graph Engine:

- Usa **Graph<int>** como grafo de interferência
- Cada vértice representa o ID de uma Web

03 Interface do Projeto

Batch Mode:

Execução automática e reprodução de resultados

```
~/Documents/2025-26/DA/Projeto2 main ?1 > ./myProg -  
isters/registers2.txt out.txt  
Allocation: FEASIBLE (ok)  
webs:      3  
registers: 2  
  r1 -> web0  
  r0 -> web1  
  r0 -> web2
```

Interactive Mode:

Configuração, parsing e execução pelo menu

```
=== Compiler Register Allocation Tool ===  
Interactive mode. Type 0 to exit.  
  
1) Set ranges file  
2) Set registers file  
3) Set output file  
4) Show current settings  
5) Load / parse input files  
6) View parsed data  
7) Run register allocation  
0) Exit  
Choice> █
```

04 Construção das Webs

Ideia principal:

Cada live range gera uma **SubWeb**

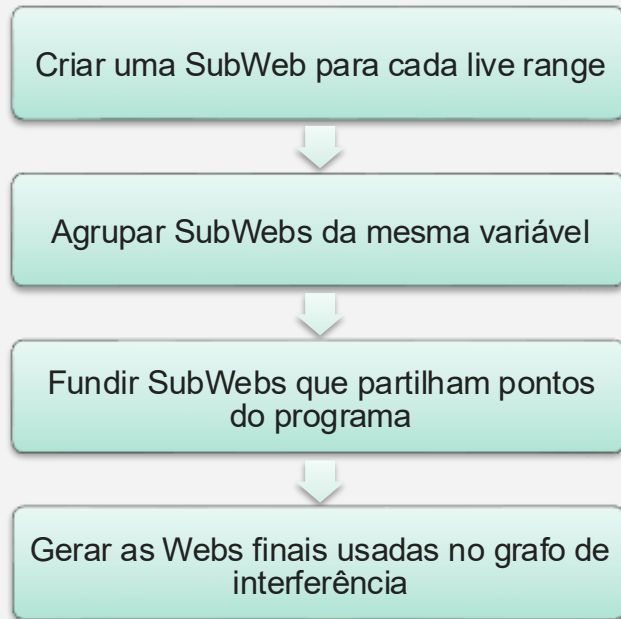
Técnica utilizada:

Union-Find para fundir **SubWebs** de forma eficiente

Porque é importante?

Manter as **SubWebs** permite aplicar splitting mais tarde

Processo:



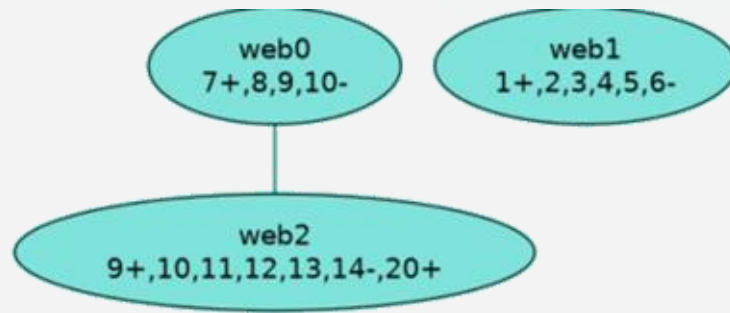
05 Grafo de Interferência

Ideia principal:

- Nós – Webs
- Arestas – Interferência
- Webs interferentes não podem partilhar o mesmo registo

Regra:

Duas Webs coexistem → não podem partilhar registo



Consequência:

Alocação de registos = coloração do grafo de interferência

06 Algoritmo Básico

Ideia:

Colorir o grafo com até **K** registros

Regras:

- Cor = registo físico
- Webs adjacentes não podem ter a mesma cor
- Se falhar → **INFEASIBLE**

Limitação:

Não faz **spilling** nem **splitting**

```
danielramaa@danielramaa
~/Doc/2025-26/DA/Projeto2 main ?2 > ./myProg -b
g_basic_K2.txt out.txt
Allocation: INFEASIBLE (basic: cannot colour in
webs:      5
registers: 0
M  -> web0
M  -> web1
M  -> web2
M  -> web3
M  -> web4
Warning: register allocation infeasible - webs
~/Documents/2025-26/DA/Projeto2 main !6 ?2 > 
```

07 Resultados nos Datasets do Enunciado

Caso	Webs	Edges	Registos	Splits	Spills	Estado
ranges1 + K=2	3	1	2	0	0	OK
ranges2 + K=2	2	1	2	0	0	OK
ranges3 + K=2	2	1	2	0	0	OK
ranges4 + K=1	4	0	1	0	0	OK
ranges5 + K=1	3	0	1	0	0	OK
ranges6 + K=3	5	5	3	0	0	OK

Conclusão:

Todos os datasets standard foram resolvidos com o número mínimo de registos

Configuração:

- Algoritmo: **basic**
- K = valor de cada ficheiro **registers*.txt**

Observações:

- **ranges4** e **ranges5** → 0 arestas
- **ranges6** → **5-cycle (cromático = 3)**
- Splits e spills sempre 0

07 Spilling

Ideia:

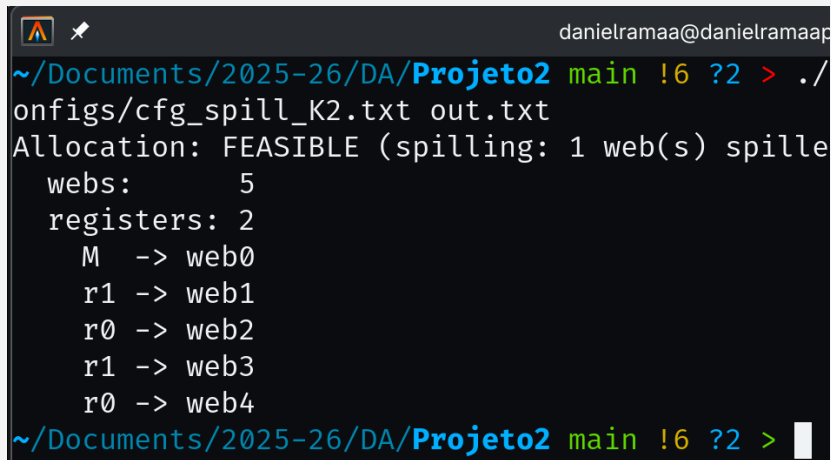
Mover uma Web para memória para simplificar o grafo

Critério:

Escolher a Web com maior grau, pois remove mais interferências

Resultado observado:

Com 1 spill, o grafo passa a ser colorível com 2 registos



```
danielramaa@danielramaap
~/Documents/2025-26/DA/Projeto2 main !6 ?2 > ./
onfigs/cfg_spill_K2.txt out.txt
Allocation: FEASIBLE (spilling: 1 web(s) spille
webs:      5
registers: 2
M  -> web0
r1 -> web1
r0 -> web2
r1 -> web3
r0 -> web4
~/Documents/2025-26/DA/Projeto2 main !6 ?2 > 
```

08 Splitting

Ideia:

Dividir uma **Web** nas suas **SubWebs**

Efeito:

Reduz interferências e reconstrói o grafo

Resultado observado:

Com 1 split, o grafo passa a ser colorível sem enviar Webs para memória

```
danielramaa@danielramaa:~$  
~/Documents/2025-26/DA/Projeto2 main !6 ?2 > ./onfigs/cfg_split_K2.txt out.  
Allocation: FEASIBLE (splitting: 1 web(s) split  
webs:      6  
registers: 2  
  r1 -> web0  
  r0 -> web1  
  r1 -> web2  
  r0 -> web3  
  r1 -> web4  
  r0 -> web5  
~/Documents/2025-26/DA/Projeto2 main !6 ?3 >
```

09 Algoritmo Híbrido

Estratégia:

Escolher a Web ativa com maior grau

- Se tem várias SubWebs → split
- Caso contrário → spill

Resultado observado:

ranges_split_clean: 1 split e 0 spills,
evitando enviar Webs para memória

```
~/Documents/2025-26/DA/Projeto2 main !6 ?3 > .  
onfigs/cfg_free_K2.txt out.txt  
Allocation: FEASIBLE (free: 1 split(s), 0 spil  
webs:      6  
registers: 2  
  r1 -> web0  
  r0 -> web1  
  r1 -> web2  
  r0 -> web3  
  r1 -> web4  
  r0 -> web5  
~/Documents/2025-26/DA/Projeto2 main !6 ?3 >
```

10 Conclusão

A estratégia híbrida combina splitting e spilling para encontrar uma alocação válida com os registos disponíveis.

Adaptativo

Modular

Eficiente